Changing 3 Service Meshes in 7 years



Igor Baliuk

Platform Engineer @ Avito

About me

Igor Baliuk

- ❏ Senior Platform Engineer
- ❏ Observability, Service Mesh



Service Mesh is not a destination

It’s difficult to stop when you start

What is a Service Mesh?

What is a Service Mesh?

A framework to organize communication between different system components

- Flexible and configurable
- Observable
- Security



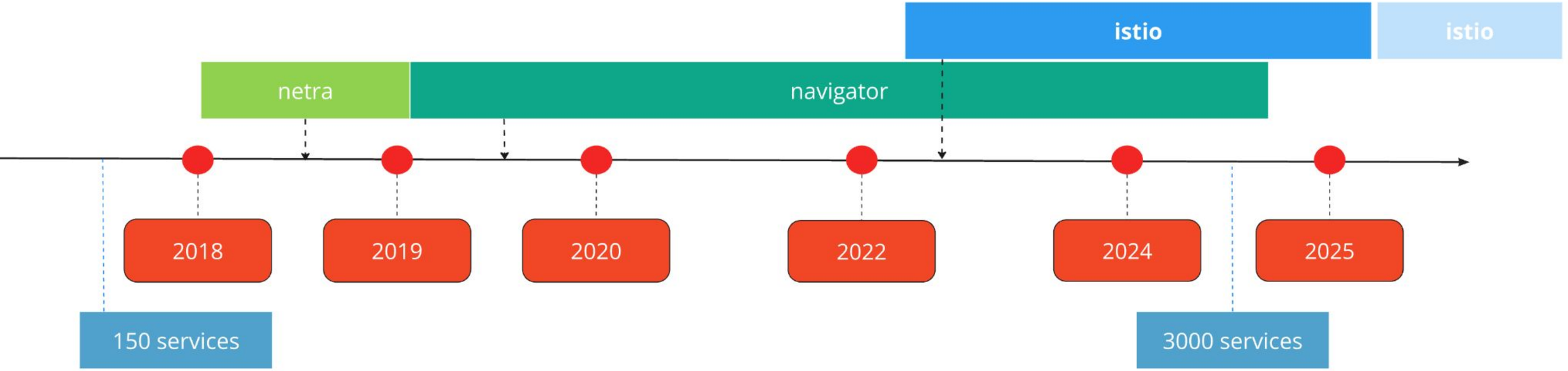
Service Mesh at Avito

- E-commerce with over 50m MAU
- > 1,500 engineers
- > 3,000 services
- > 100k pods
- > 1m RPS
- On-prem and a bit of cloud



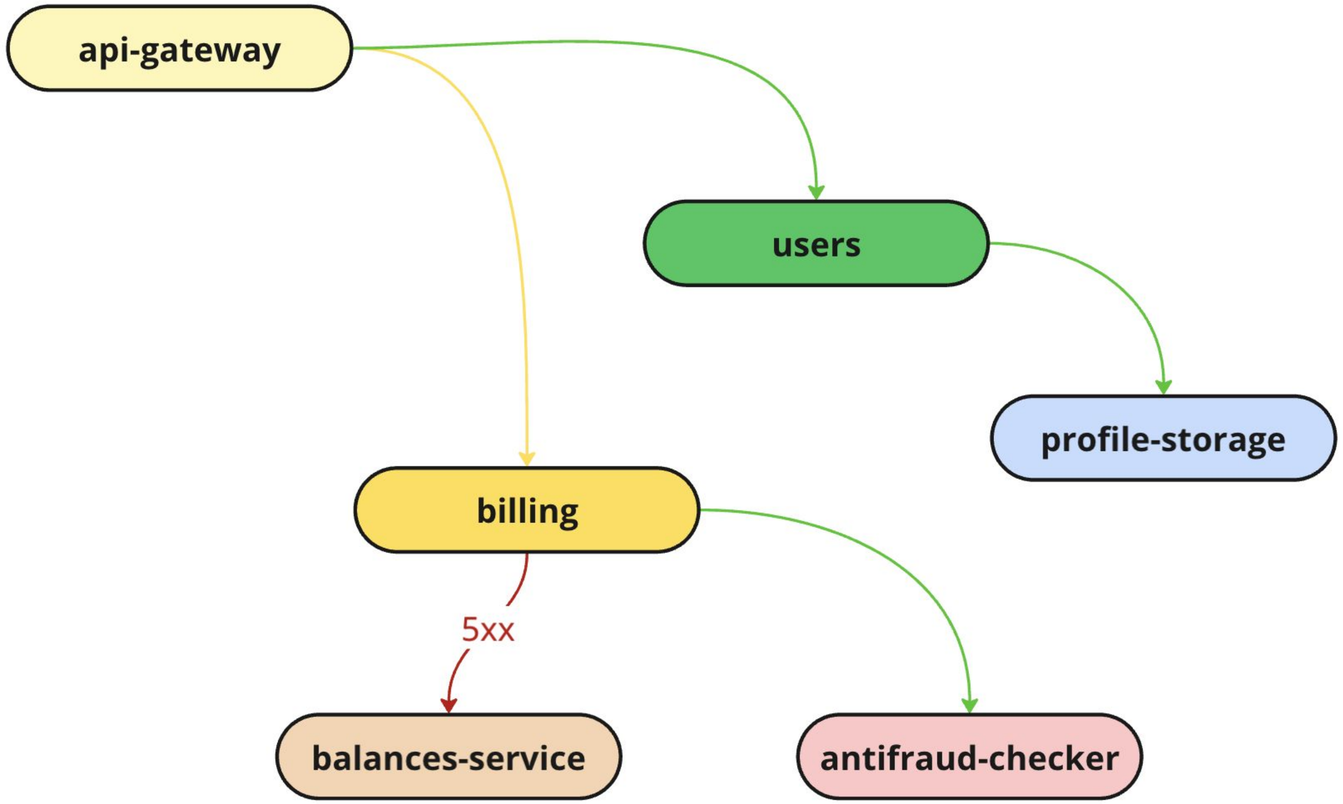
How we started

Evolution

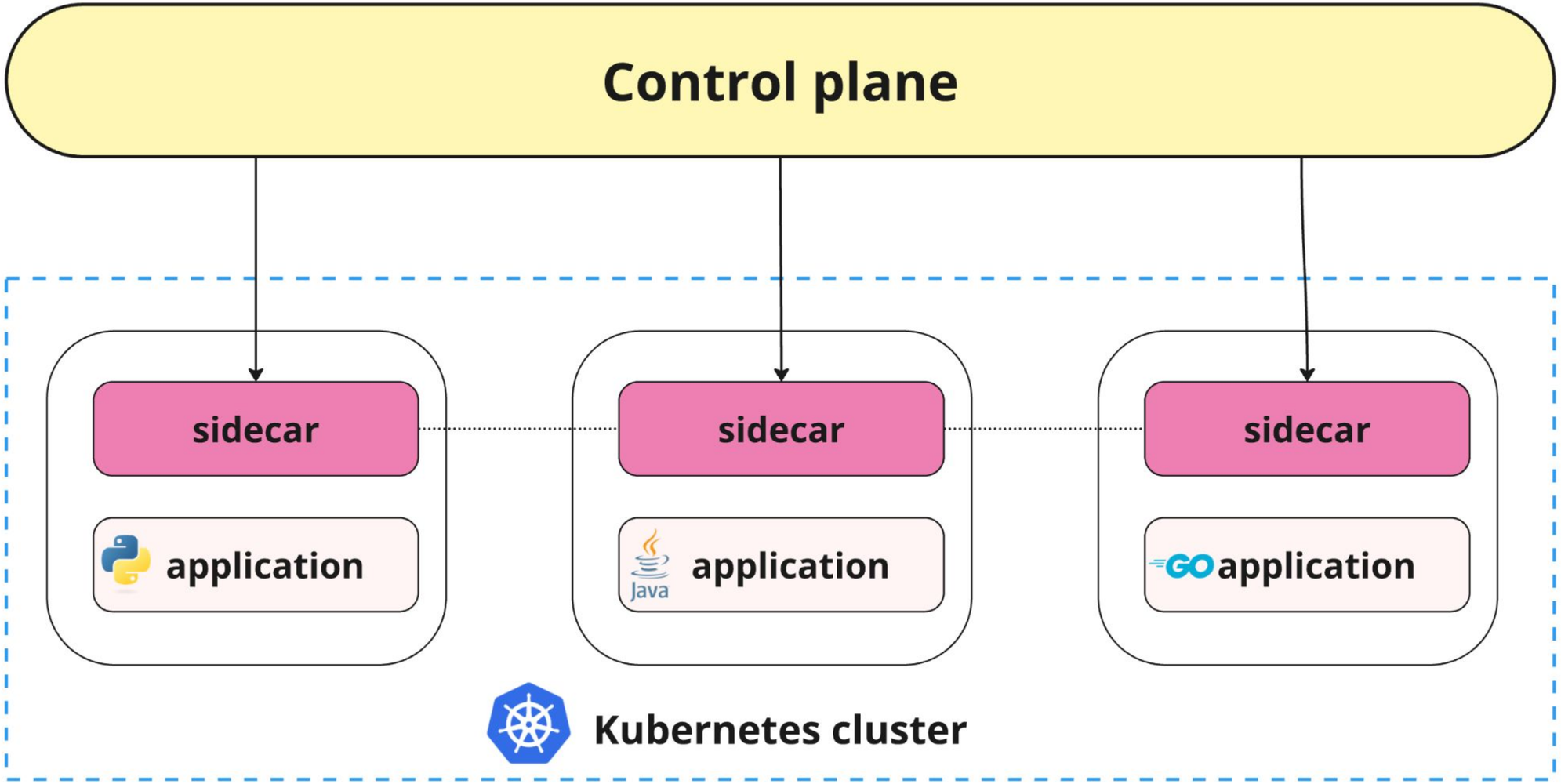

istio
istio
netra
navigator
2018
2019
2020
2022
2024
2025
150 services
3000 services

2018

- > 150 services
- Several Kubernetes clusters
- Visualizing the service map



2018: a typical service mesh



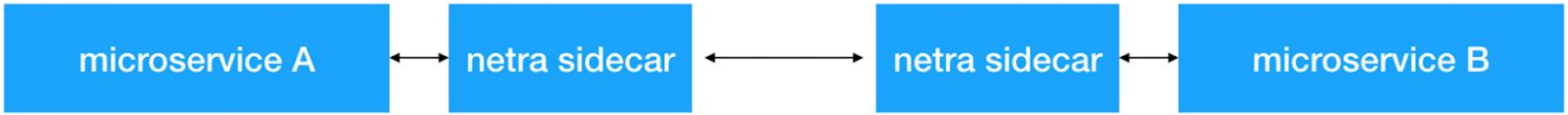
2018: a typical service mesh

- Istio and Linkerd existed
- But we were afraid of complexity and overhead

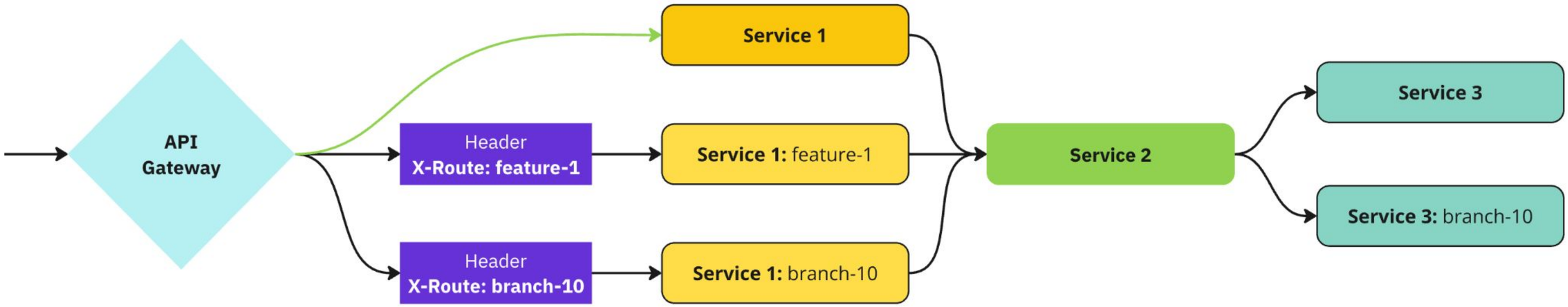
First attempt: netra, designed by us

Fast and simple proxy

- < 2,000 lines of code in Go
- < 50 MB RAM
- Metrics and tracing
- Basic traffic rerouting



Lightweight isolated environments for simultaneous testing



netra

+ Better Observability
+ Simplified Testing
+ Boosted TTM

netra

+ Better Observability
+ Simplified Testing
+ Boosted TTM

– No service discovery
– No flexibility
– Cross-cluster communication is messy
– Safety?

We liked the idea

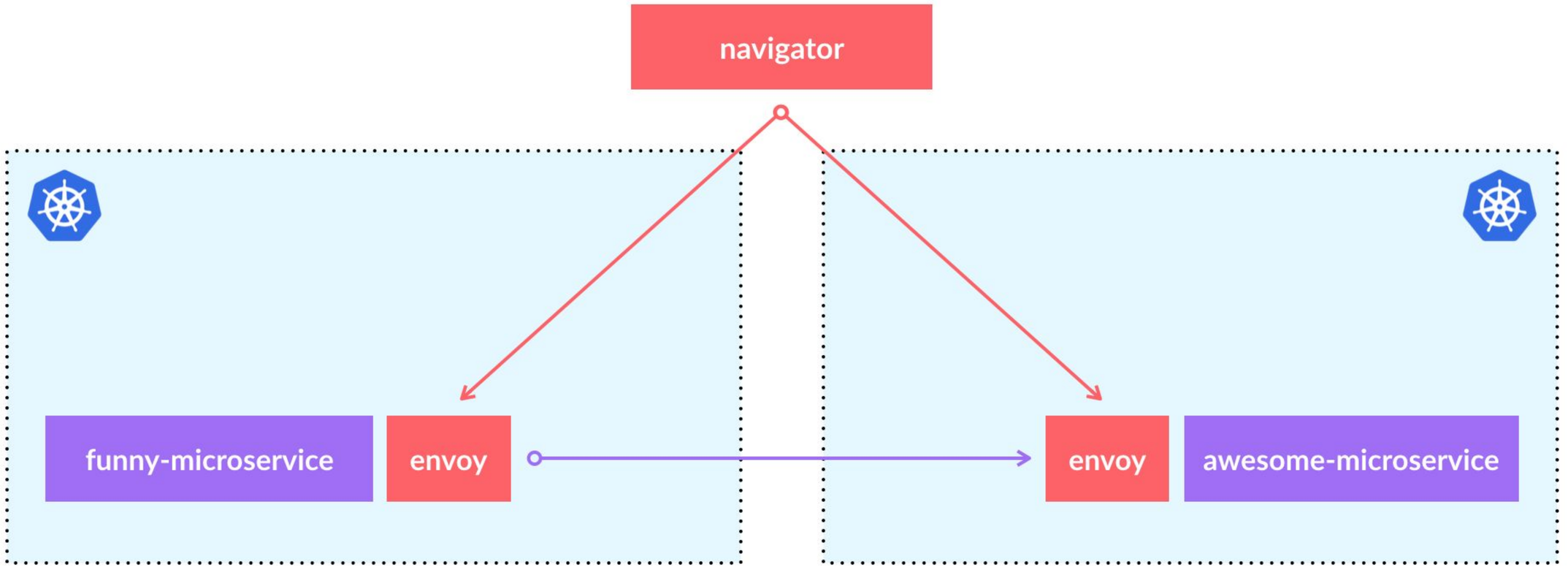
Let’s try something bigger

2019: trying Istio

- Performance issues
- Each instance was aware of all other instances
- Cross-cluster communication was problematic
- We encountered bugs



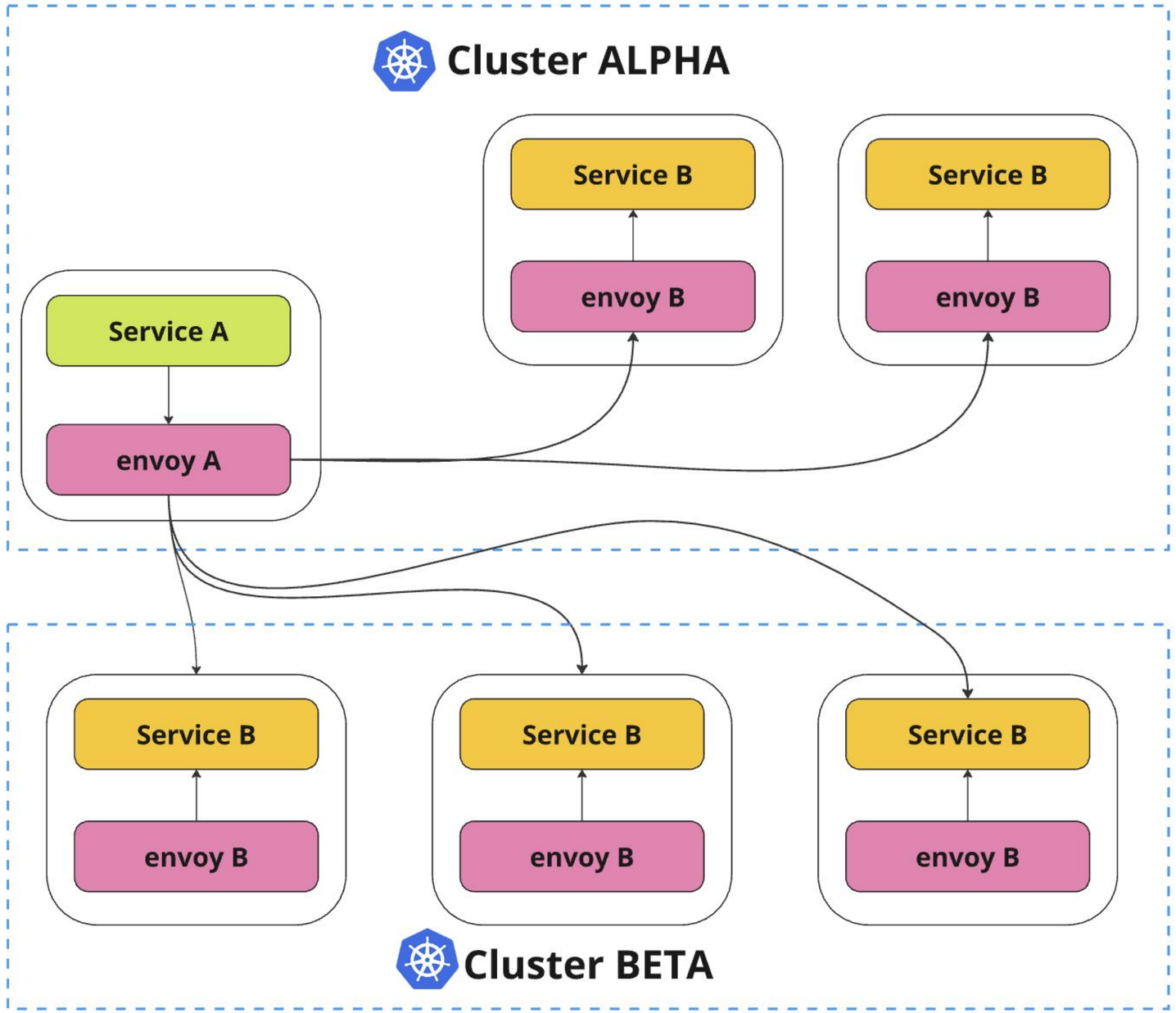
One more attempt: Navigator



- Inspired by Project Contour
- Implement only what is truly necessary

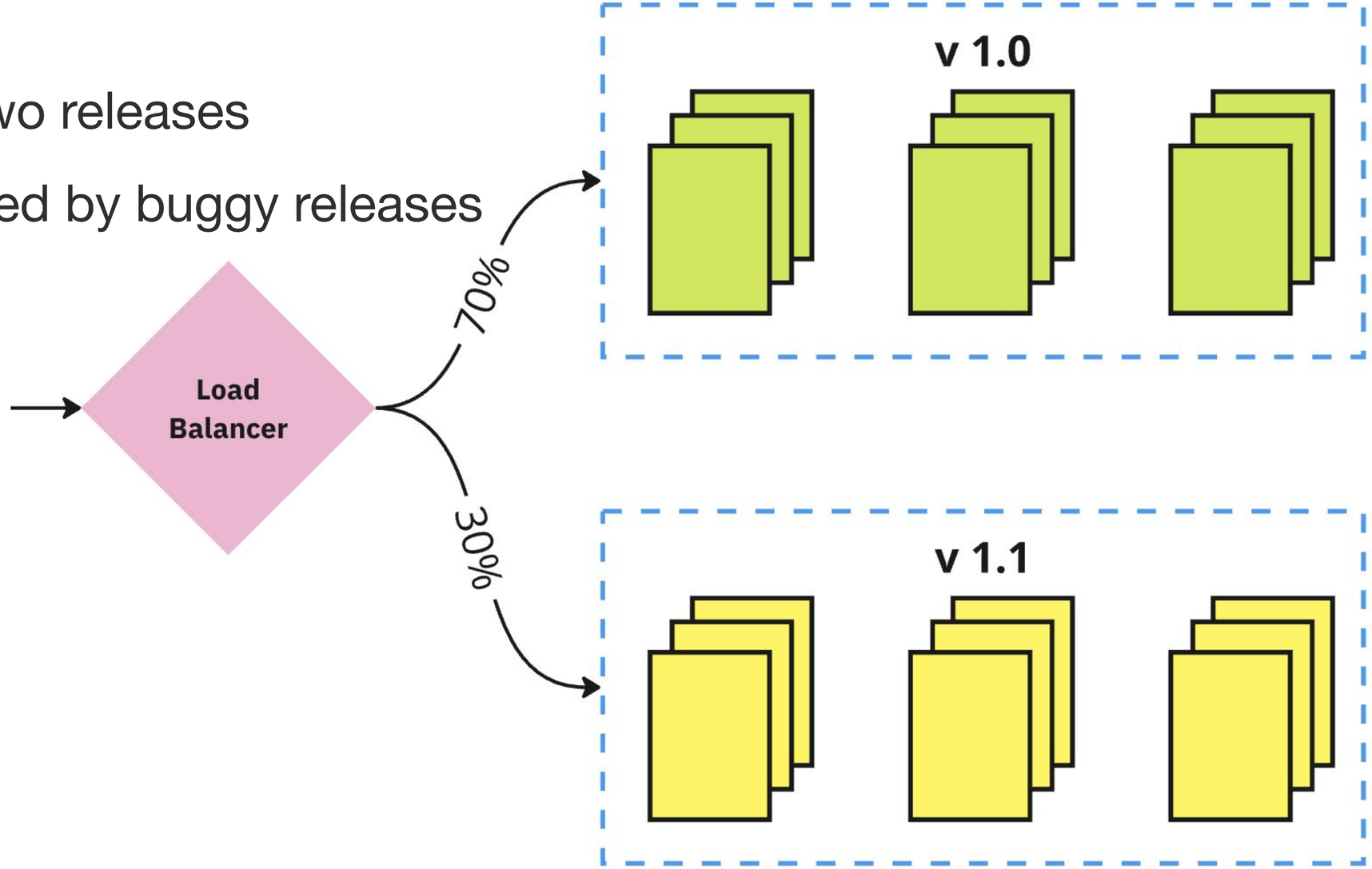
Service Discovery

- Client simplicity
- Fault tolerance
- Traffic locality



Canary Releases

- Split traffic between two releases
- Reduce damage caused by buggy releases



Simple Configuration

- Existed: Service and Ingress
- New: CanaryRelease and Nexus

```yaml
apiVersion: v1
items:
- apiVersion: navigator.avito.ru/v1
  kind: Nexus
  metadata:
    ...
  spec:
    appName: cars-service-catalog-2119-staging
    cookieAffinity: ""
    downstreams:
    - name: cars-service-catalog-2119-staging
      namespace: cars-service
    headerAffinity: ""
    inboundPorts:
    - name: http
      port: 8890
    mTLS:
      enabled: true
      spiffeID: spiffe://company/ns/cars-service/service/non-prod
    rateLimit:
      enabled: false
      port: 15011
    serviceName: cars-service
    services:
    - namespace: toggles
    - namespace: geo-locator
    - namespace: image-classifier
    - namespace: ab-tests
    - namespace: delivery-layout
    - namespace: cars-storage
    - namespace: user-profile
    - namespace: cars-service
```

It’s time to think about Safety

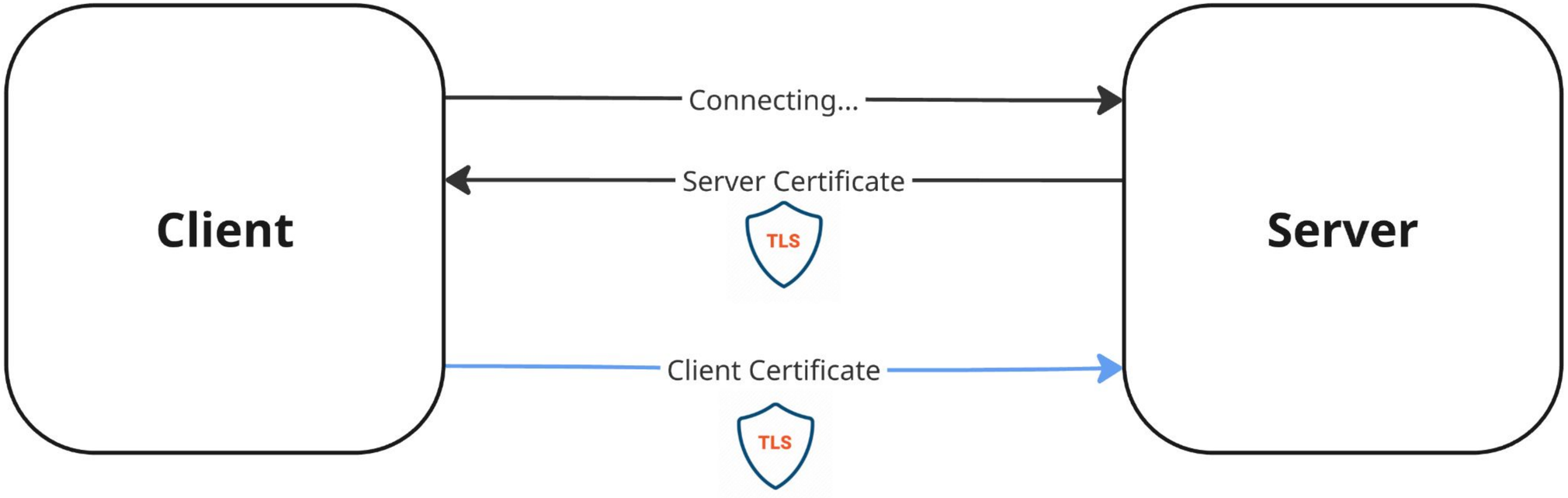
HTTPS (TLS)

+ Network Encryption
+ Easy to enable

– Clients are not authenticated

Mutual TLS (mtLS)

Now clients have certificates



mTLS

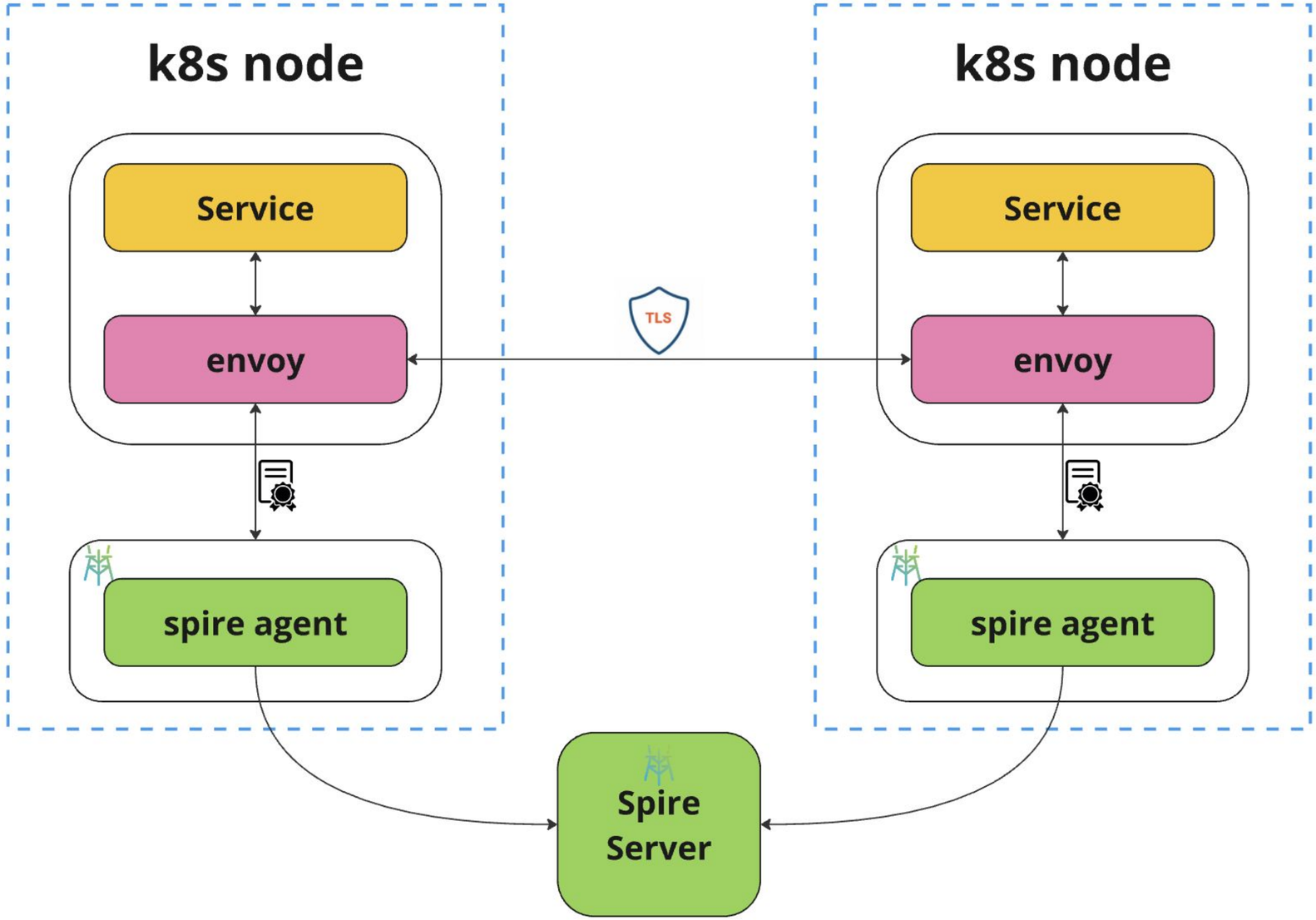
- An attacker also needs a certificate
- Authorization policies

```
[clients]
defaultClients = ["service:kratos"]

[[policy]]
endpoints = [
    "rpc:getItems",
    "rpc:updateItem",
]
clients = [
    "service:item-processor",
    "$defaultClients",
]

[[policy]]
endpoints = [
    "http:/admin/users/block",
    "http:/admin/users/remove",
]
clients = [
    "team:$service.team",
    "service:admin-gateway",
]
```

mTLS: architecture



mTLS: Disaster Recovery Plan

- mTLS has an enormous blast radius
- One day, **it will break**




```
x509: certificate has expired or
      is not yet valid
```

Developing a Service Mesh from scratch is *difficult*

- A team of four engineers
- Testing is difficult
- Afraid to roll out new control plane versions

New requirement: cloud integration

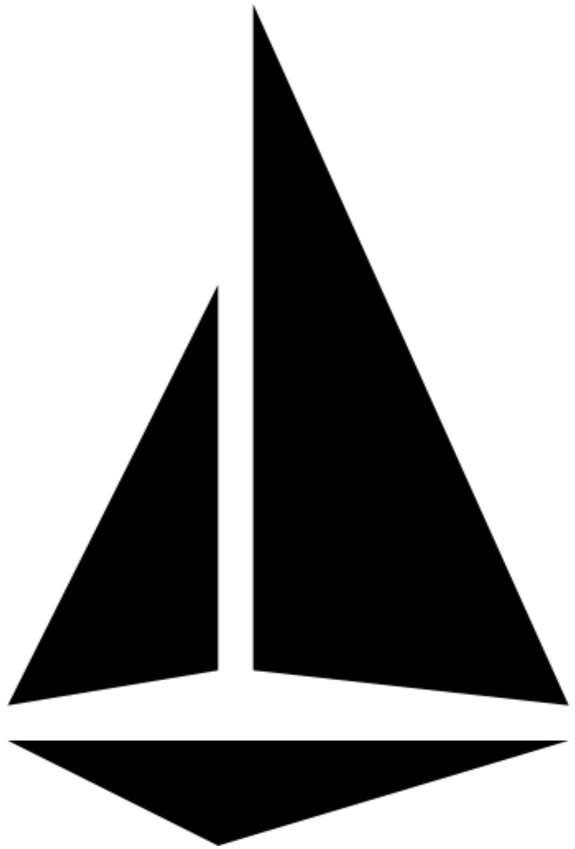


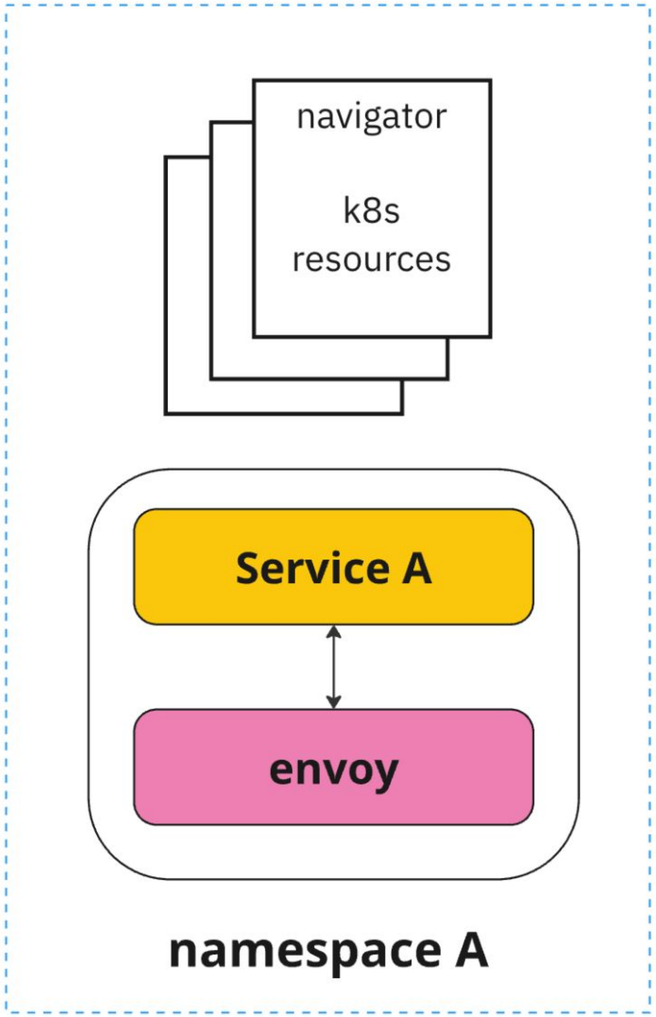
The final boss

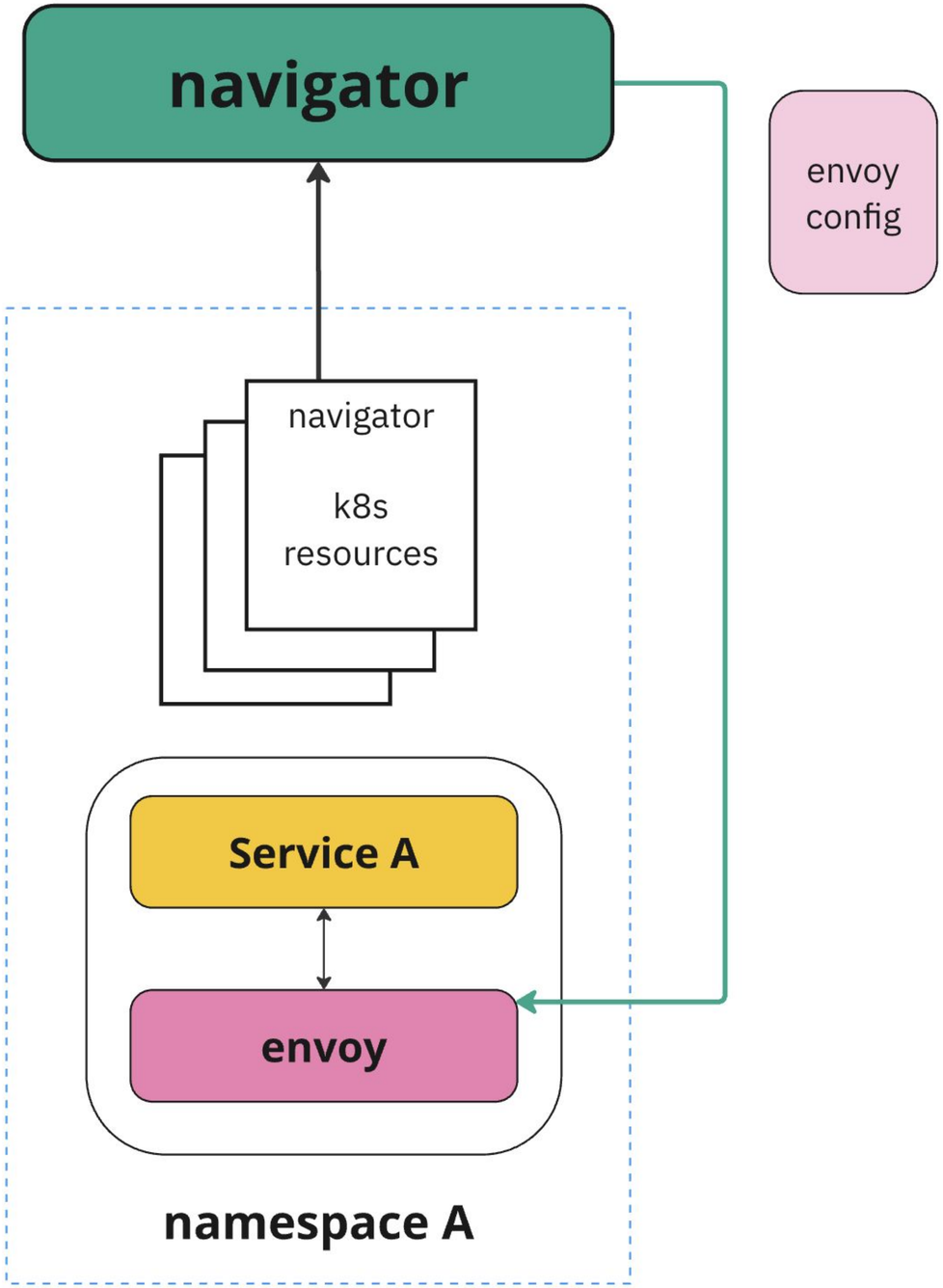
Istio

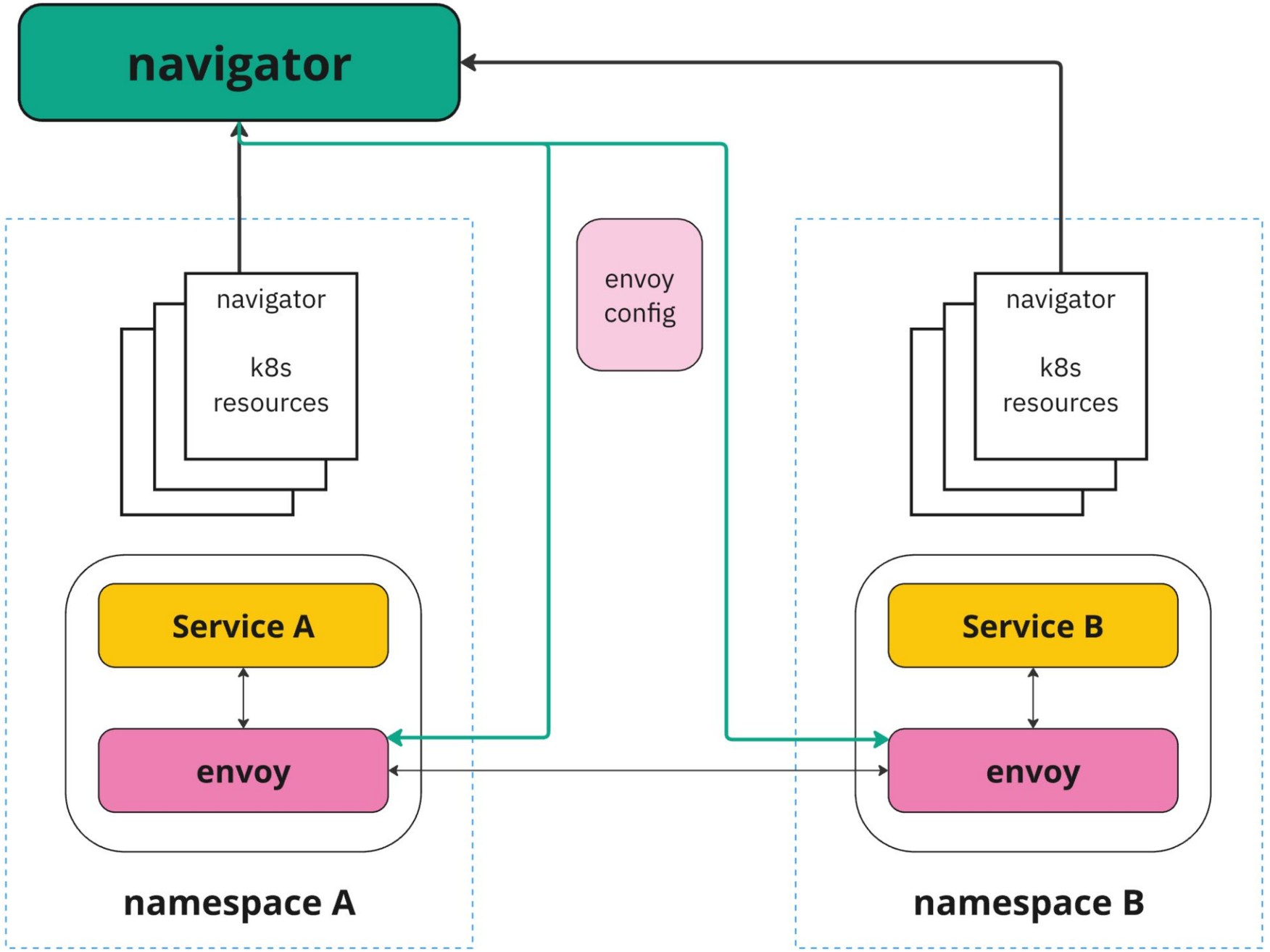
2022: Istio

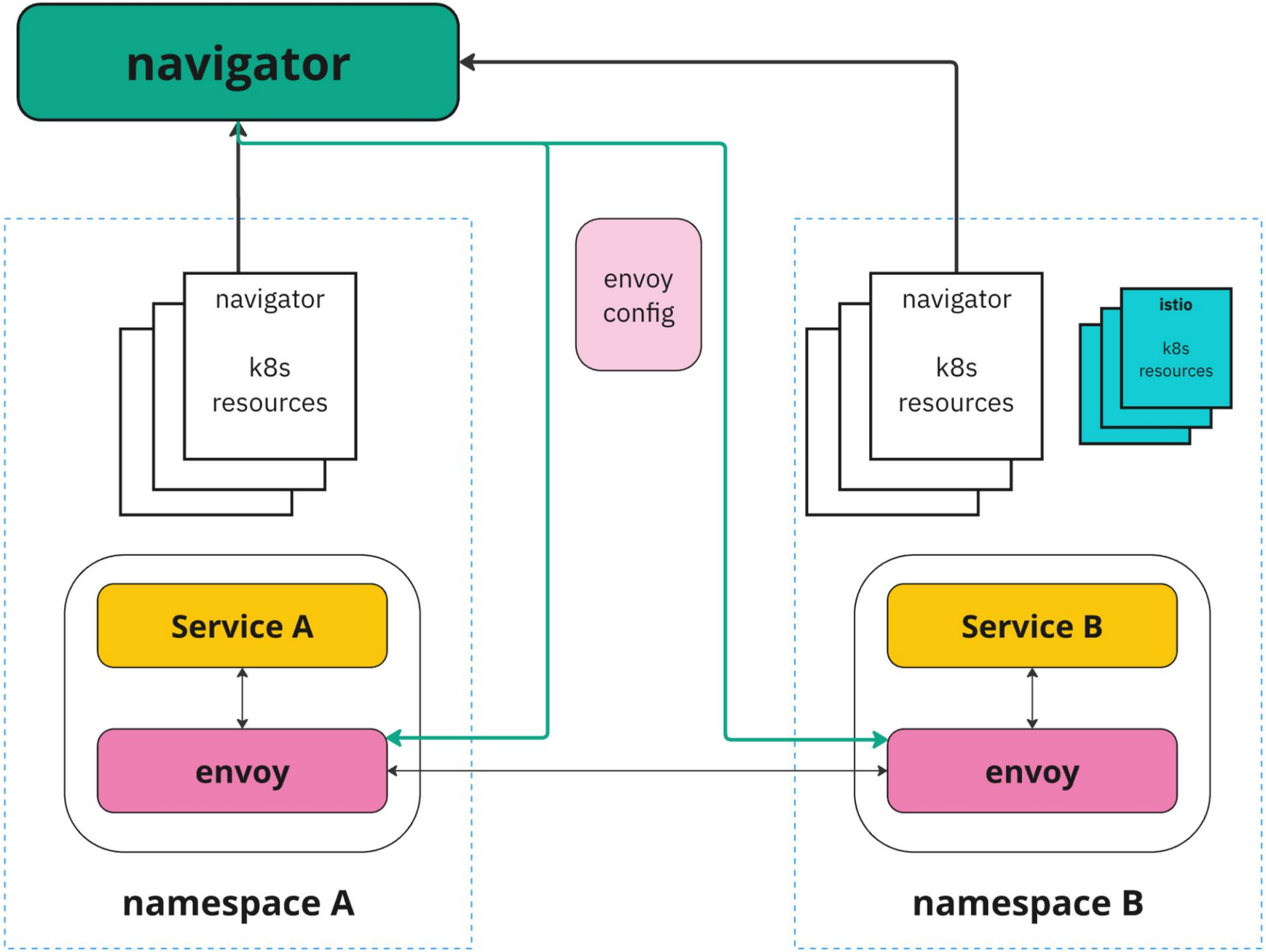
- We had Envoy
- Istio became stable
- Istio is tested by other companies
- Documentation
- Hybrid-cloud solutions (east-west gateway)

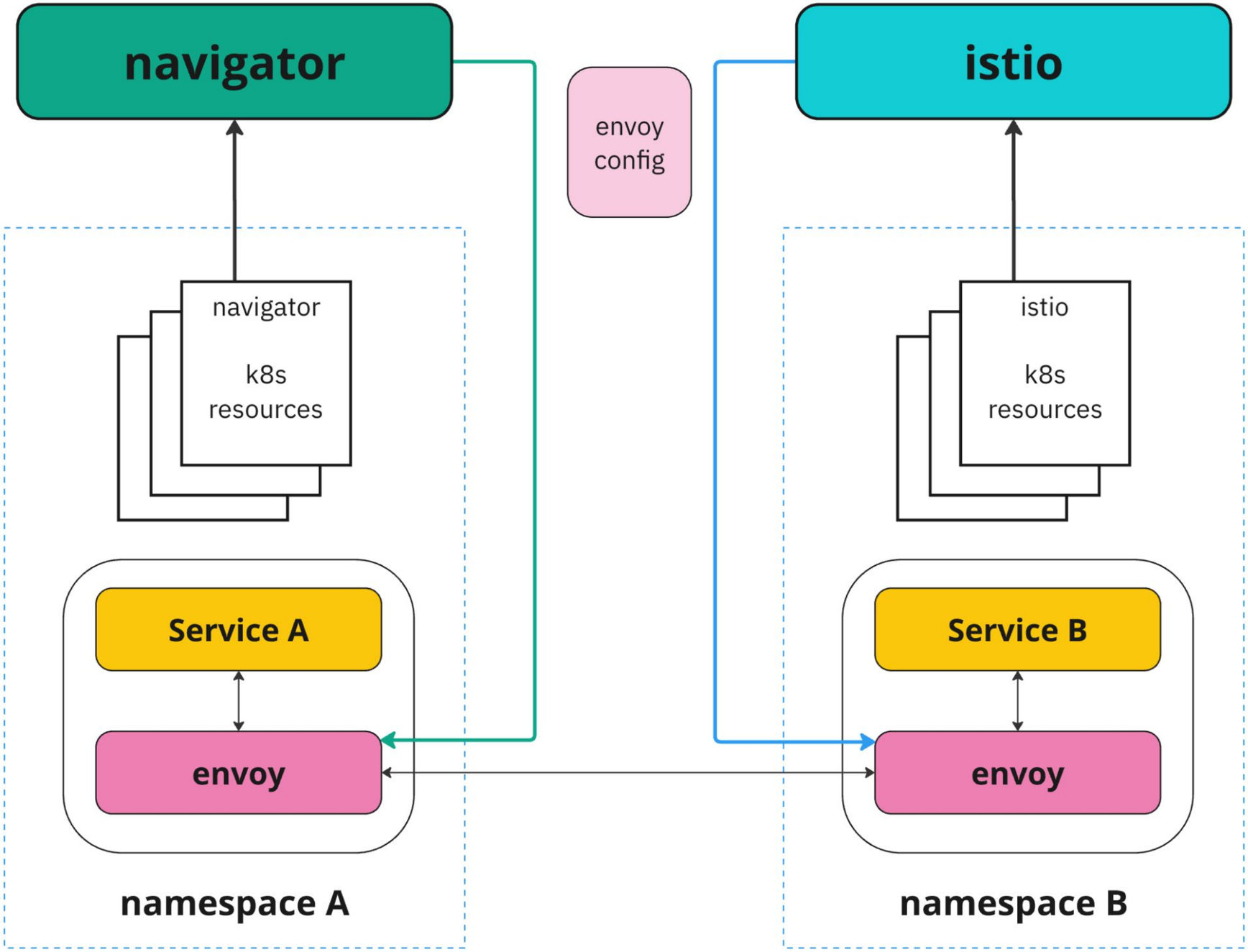



navigator
k8s
resources
Service A
envoy
namespace A


navigator
envoy
config
navigator
k8s
resources
Service A
envoy
namespace A


navigator
navigator
k8s
resources
envoy
config
navigator
k8s
resources
Service A
envoy
Service B
envoy
namespace A
namespace B


navigator
navigator
k8s
resources
envoy
config
navigator
k8s
resources
istio
k8s
resources
Service A
envoy
Service B
envoy
namespace A
namespace B


navigator
istio
envoy
config
navigator
k8s
resources
istio
k8s
resources
Service A
Service B
envoy
envoy
namespace A
namespace B

How to accelerate transition?

Developers do not have direct access to Kubernetes manifests.

No access to manifests

- ❏ Product engineers write business-related code
- ❏ Manifests are managed by the platform (infra) teams

No access to manifests

- ❏ Product engineers write business-related code
- ❏ Manifests are managed by the platform (infra) teams
- ❏ Services are managed via IDP



Istio and Its Challenges

Wide range of Kubernetes versions

- ❏ Different Kubernetes versions: 1.14 – 1.20+
- ❏ Different versions have different resource APIs

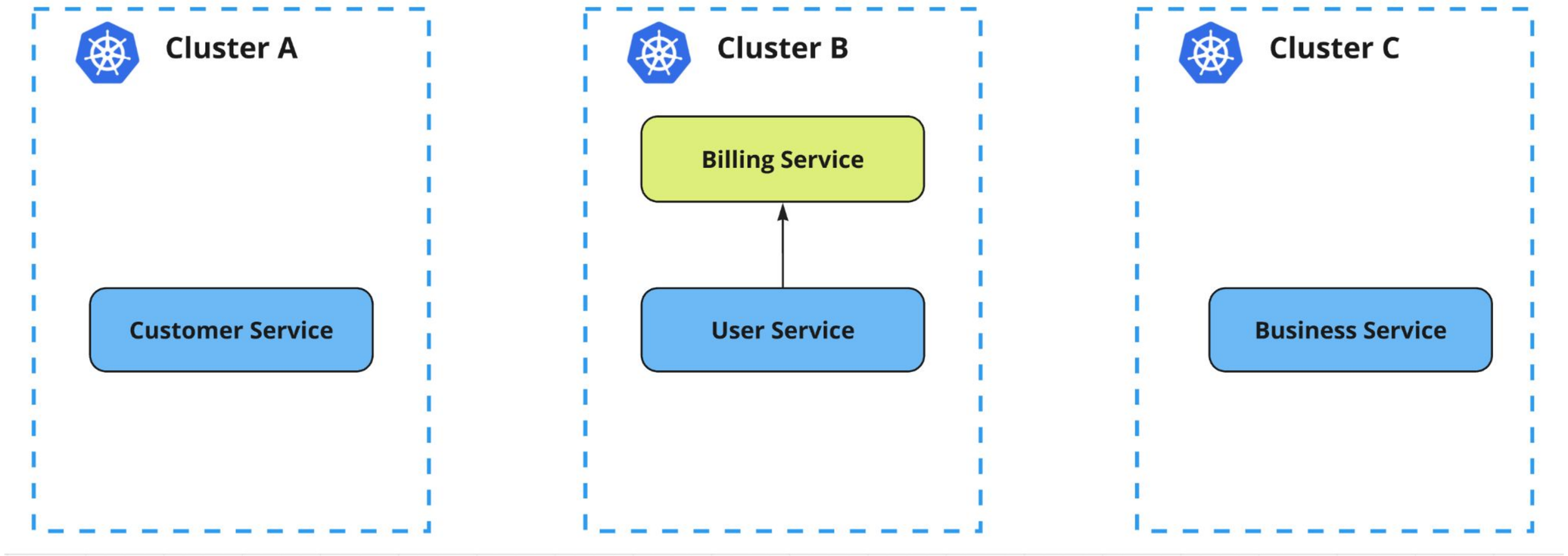


Fork Relevance

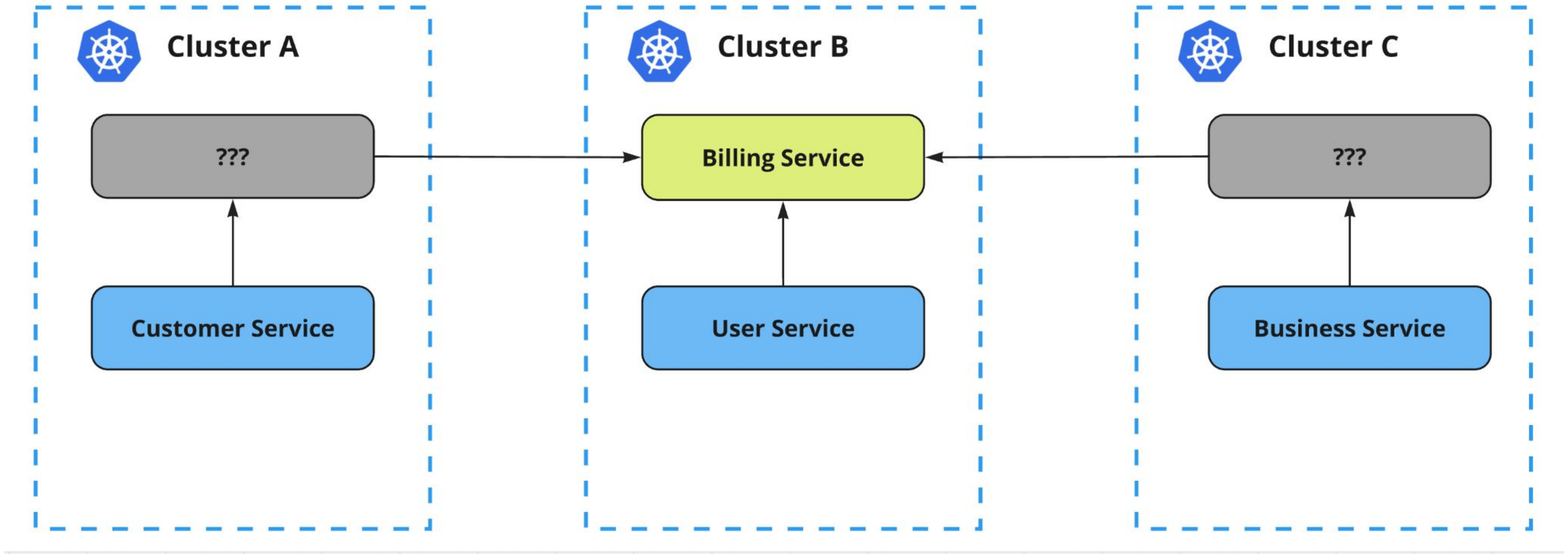
- ❏ Keep a small set of changes
- ❏ Rare upgrades
- ❏ Upgrade Istio along with Kubernetes

Version	Currently Supported	Release Date	End of Life	Supported Kubernetes Versions	Tested, but not supported
master	No, development only			1.29, 1.30, 1.31, 1.32	1.23, 1.24, 1.25, 1.26, 1.27, 1.28
1.25	Yes	March 03, 2025	~Dec 2025 (Expected)	1.29, 1.30, 1.31, 1.32	1.24, 1.25, 1.26, 1.27, 1.28
1.24	Yes	November 7, 2024	~Aug 2025 (Expected)	1.28, 1.29, 1.30, 1.31	1.23, 1.24, 1.25, 1.26, 1.27
1.23	Yes	Aug 14, 2024	Apr 16, 2025	1.27, 1.28, 1.29, 1.30	1.23, 1.24, 1.25, 1.26
1.22	No	May 13, 2024	Jan 22, 2025	1.27, 1.28, 1.29, 1.30	1.23, 1.24, 1.25, 1.26
1.21	No	Mar 13, 2024	Sept 27, 2024	1.26, 1.27, 1.28, 1.29	1.23, 1.24, 1.25
1.20	No	Nov 14, 2023	Jun 25, 2024	1.25, 1.26, 1.27, 1.28, 1.29	1.23, 1.24

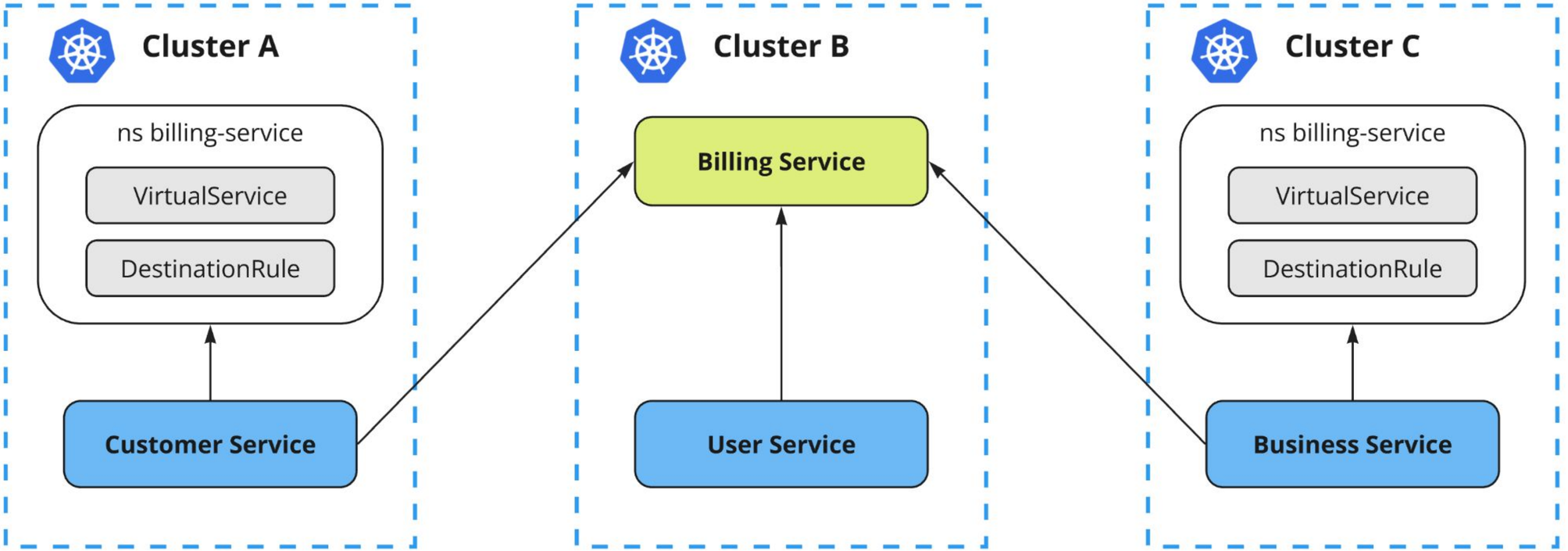
Cross-cluster communication



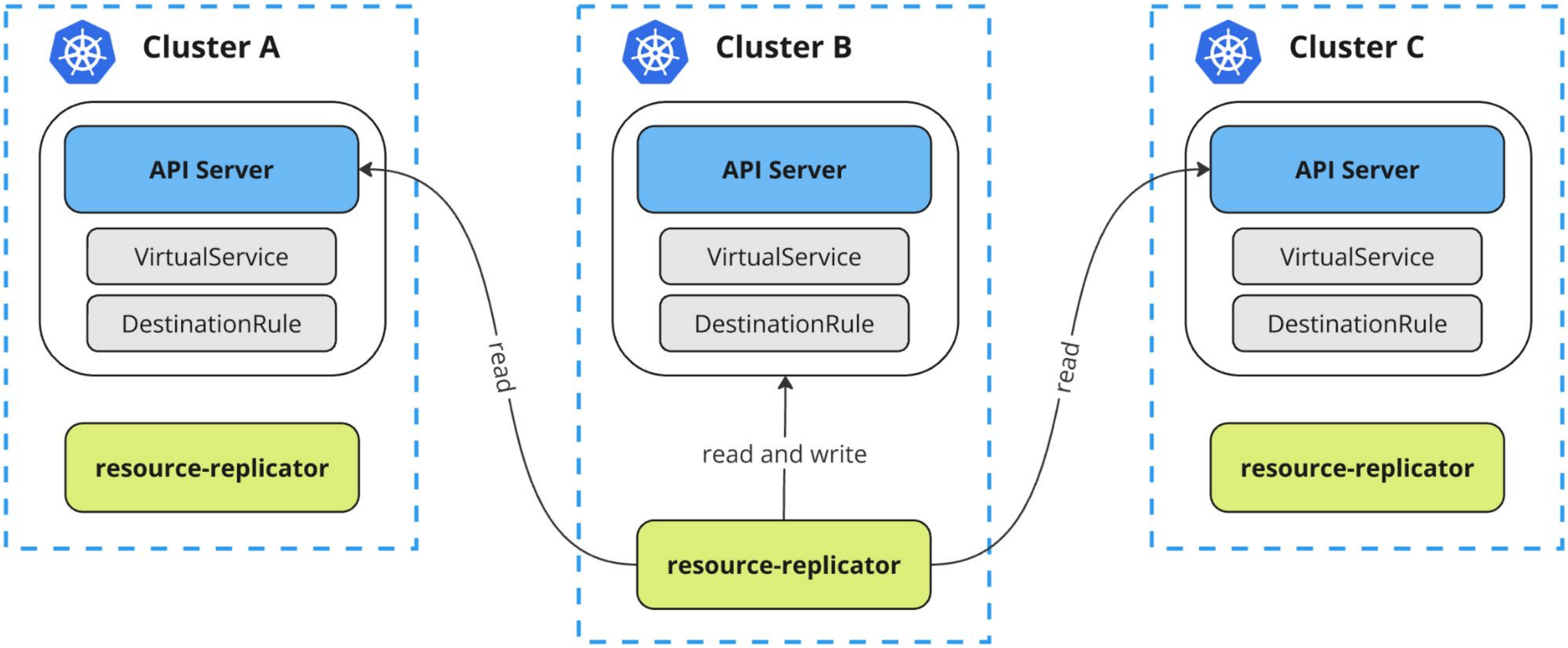
Cross-cluster communication



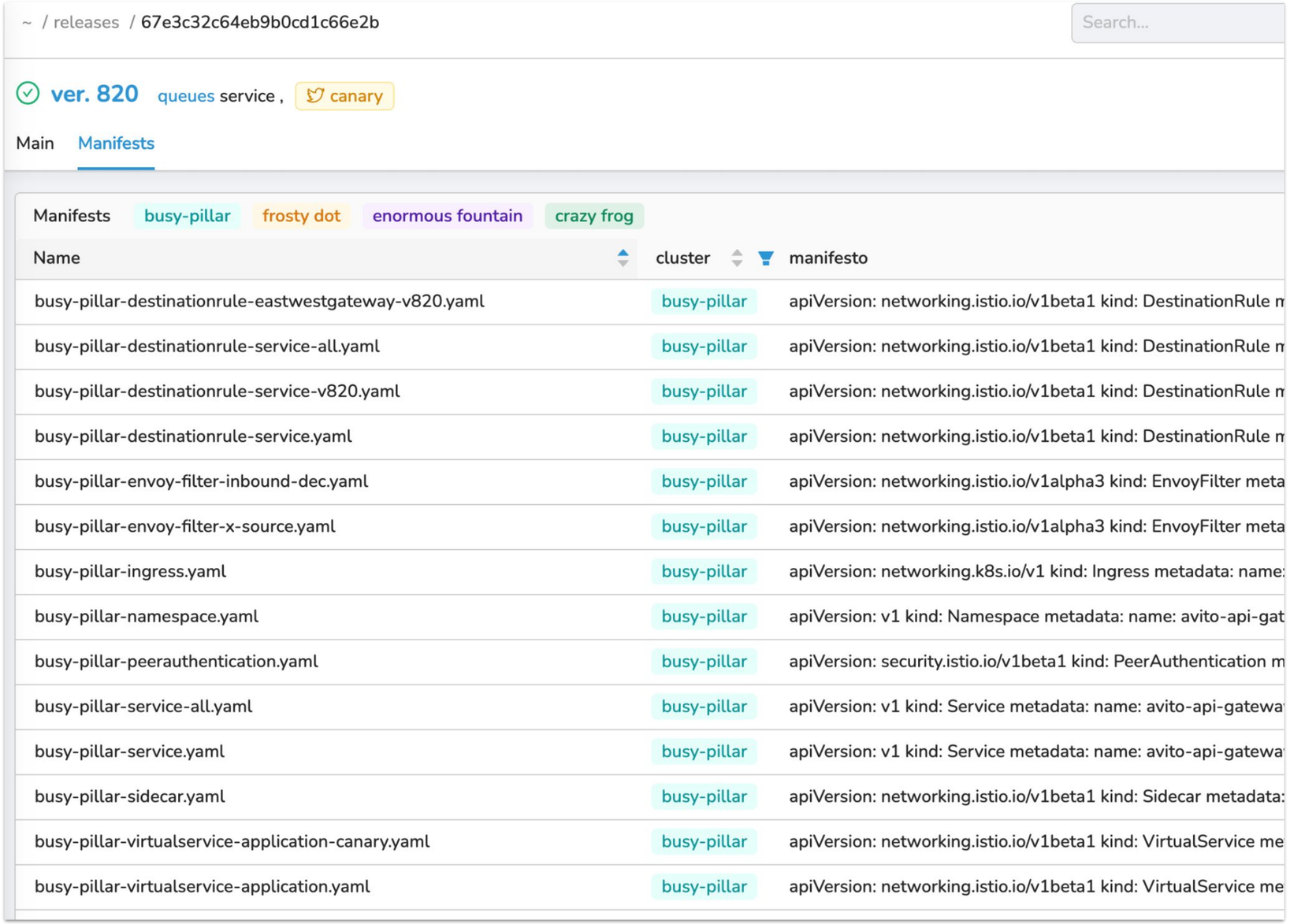
Cross-cluster communication: VS and DR



VS and DR must be presented even if a deployment is missing



Loads of manifests…



Debugging Istio

“Why is the latency so high”?

“I see upstream connect errors…”

- ❏ Conflicting configuration
- ❏ Long configuration update dissemination

Config Updates Dissemination

Expected order:

1. Roll out a new deployment version
2. Configure the mesh for the new deployment
3. Route traffic to the new deployment
4. Scale down the old deployment

Config Updates Dissemination

Unexpected order:

1. Roll out a new deployment version
3. Route traffic to the new deployment
2. Configure the mesh for the new deployment
4. Scale down the old deployment

Oops, traffic is going to nowhere…



Debugging Istio

Tools

- `tcpdump`
- `istioctl`
- `envoy config_dump`
- `strace, ebpf, …`

Metrics

- `istio_requests_total`
- `pilot_xds_*`



Sidecar Footprint

P99 **1.0** CPU **350** **MiB** RAM
P75 **0.2** CPU **170** **MiB** RAM



The end

Resume

- ❏ Service Mesh is difficult
- ❏ It breaks
- ❏ It’s complex
- ❏ You need a team to maintain it
- ❏ There is no silver bullet



Service Mesh is Great

- ❑ Canary rollouts
- ❑ Fault tolerance
- ❑ Flexible traffic management
- ❑ Observability
- ❑ Security



Service Mesh is Great

- ❑ Canary rollouts
- ❑ Fault tolerance
- ❑ Flexible traffic management
- ❑ Observability
- ❑ Security
- ❑ **Enables scaling**



And one team is enough to maintain it

Thank You!

Should developers write manifests?



Igor Baliuk

baliuk.igor@gmail.com

linkedin.com/in/baliukigor/